

MATH00041: Introduction to Programming

Dictionaries and Sets

Lecture 5

Archontis Giannakidis, PhD

Outline

- **Dictionary**
- **Sets**

Python Dictionaries

Python's dictionary is a data structure that stores data values as key-value pairs, similar to a map. Unlike other data types that can hold only a single value, a dictionary can contain multiple key-value pairs, making it a more efficient way to store data.

```
thisdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}
```

From Python's perspective, dictionaries are defined as objects with the data type 'dict':

```
<class 'dict'>
```

Dictionaries store data in key: value pairs.

Dictionaries are written with curly brackets, and have keys and values:

Example

Create and print a dictionary:

```
thisdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
print(thisdict)
```

Output: {'brand': 'Ford', 'model': 'Mustang', 'year': 1964}

Dictionary Items

Dictionary items are presented as key-value pairs, ordered, and changeable but do not allow duplicates.

Example

Print the "brand" value of the dictionary:

```
thisdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
print(thisdict["brand"])
```

Output: Ford

Dictionaries cannot have two items with the same key:

Example

Duplicate values will overwrite existing values:

```
thisdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964,  
    "year": 2020  
}  
print(thisdict)
```

Output: {'brand': 'Ford', 'model': 'Mustang', 'year': 2020}

Dictionary Length

To determine how many items a dictionary has, use the `len()` function:

Example

Print the number of items in the dictionary:

```
print(len(thisdict))
```

Output: 3

Dictionary Items - Data Types

The values in dictionary items can be of any data type:

Example

String, int, boolean, and list data types:

```
thisdict = {  
    "brand": "Ford",  
    "electric": False,  
    "year": 1964,  
    "colors": ["red", "white", "blue"]  
}  
print(thisdict)
```

Output: {'brand': 'Ford', 'electric': False, 'year': 1964, 'colors': ['red', 'white', 'blue']}

The dict() Constructor

Furthermore, to create a dictionary, you can use the dict() constructor.

Example

Using the dict() method to make a dictionary:

```
thisdict = dict(name = "John", age = 36, country = "Norway")  
print(thisdict)
```

Output: {'name': 'John', 'age': 36, 'country': 'Norway'}

Accessing Items

You can retrieve values from a dictionary using the key name inside square brackets. The `get()` method can also be used for the same result.

Example

Get the value of the "model" key:

```
thisdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
x = thisdict["model"]  
print(x)
```

Output: Mustang

Example

Get the value of the "model" key:

```
x = thisdict.get("model")  
print(x)
```

Output: Mustang

Get Keys

The `keys()` method will return a list of all the keys in the dictionary.

Example

Get a list of the keys:

```
x = thisdict.keys()
```

```
print(x)
```

Output: dict_keys(['brand', 'model', 'year'])

The `keys()` method returns a view object containing the keys of the dictionary. Any changes made to the dictionary are reflected in the keys object.

Example

Add a new item to the original dictionary, and see that the keys list gets updated as well:

```
car = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}
```

```
x = car.keys()
```

```
print(x) #before the change
```

```
car["color"] = "white"
```

```
print(x) #after the change
```

Output: dict_keys(['brand', 'model', 'year']) #before the change

Output: dict_keys(['brand', 'model', 'year', 'color']) #after the change

Get Values

The `values()` method will return a list of all the values in the dictionary.

Example

Get a list of the values:

```
x = thisdict.values()
```

```
print(x)
```

Output: dict_values(['Ford', 'Mustang', 1964])

The values list is a view of the dictionary and reflects any changes made to it.

Example

Make a change in the original dictionary, and see that the values list gets updated as well:

```
car = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}
```

```
x = car.values()
```

```
print(x) #before the change
```

```
car["year"] = 2020
```

```
print(x) #after the change
```

```
Output: dict_values(['Ford', 'Mustang', 1964]) #before the change
```

```
Output: dict_values(['Ford', 'Mustang', 2020]) #after the change
```

Example

Add a new item to the original dictionary, and see that the values list gets updated as well:

```
car = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}
```

```
x = car.values()
```

```
print(x) #before the change
```

```
car["color"] = "red"
```

```
print(x) #after the change
```

```
Output: dict_values(['Ford', 'Mustang', 1964]) #before the change
```

```
Output: dict_values(['Ford', 'Mustang', 1964, 'red']) #after the change
```

Get Items

The method `items()` return a list of tuples containing a dictionary's key-value pairs.

Example

Get a list of the key:value pairs

```
x = thisdict.items()
```

```
print(x)
```

Output: dict_items([('brand', 'Ford'), ('model', 'Mustang'), ('year', 1964)])

The dictionary's items list is a view of the dictionary itself, so any changes to the dictionary will be reflected in the items list.

Example

Make a change in the original dictionary, and see that the items list gets updated as well:

```
car = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}
```

```
x = car.items()
```

```
print(x) #before the change
```

```
car["year"] = 2020
```

```
print(x) #after the change
```

Output: dict_items([('brand', 'Ford'), ('model', 'Mustang'), ('year', 1964)]) #before the change

Output: dict_items([('brand', 'Ford'), ('model', 'Mustang'), ('year', 2020)]) #after the change

Example

Add a new item to the original dictionary, and see that the items list gets updated as well:

```
car = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}
```

```
x = car.items()
```

```
print(x) #before the change
```

```
car["color"] = "red"
```

```
print(x) #after the change
```

Output: dict_items([('brand', 'Ford'), ('model', 'Mustang'), ('year', 1964)])#before the change

Output: dict_items([('brand', 'Ford'), ('model', 'Mustang'), ('year', 1964), ('color', 'red')]) #after
the change

Check if the Key Exists

To check if a specific key exists in a dictionary, use the "in" keyword:

Example

Check if "model" is present in the dictionary:

```
thisdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
if "model" in thisdict:  
    print("Yes, 'model' is one of the keys in the thisdict dictionary")
```

Output: Yes, 'model' is one of the keys in the thisdict dictionary

Change Values

You can modify the value of a specific item by referencing its key:

Example

Change the "year" to 2018:

```
thisdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
thisdict["year"] = 2018  
print(thisdict)
```

Output: {'brand': 'Ford', 'model': 'Mustang', 'year': 2018}

Update Dictionary

The `update()` method adds items from the argument dictionary to the original dictionary. If the item already exists in the original dictionary, its value will be updated.

Example

Update the "year" of the car by using the `update()` method:

```
thisdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
thisdict.update({"year": 2020})  
print(thisdict)
```

Output: {'brand': 'Ford', 'model': 'Mustang', 'year': 2020}

Adding Items

A dictionary is a collection of key-value pairs, where each key is unique. To add a new item to a dictionary, you simply need to specify a new key and assign a value to it. This can be done using the standard syntax for dictionary assignments.

Example

```
thisdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
thisdict["color"] = "red"  
print(thisdict)
```

Output: {'brand': 'Ford', 'model': 'Mustang', 'year': 1964, 'color': 'red'}

Update Dictionary

The `update()` method adds items from the given argument to the dictionary. The argument must contain key: value pairs.

Example

Add a color item to the dictionary by using the `update()` method:

```
thisdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
thisdict.update({"color": "red"})  
print(thisdict)
```

Output: {'brand': 'Ford', 'model': 'Mustang', 'year': 1964, 'color': 'red'}

Removing Items

A dictionary can be manipulated in several ways to remove items from it.:

Example

The `pop()` method removes the item with the specified key name:

```
thisdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
thisdict.pop("model")  
print(thisdict)
```

Output: {'brand': 'Ford', 'year': 1964}

Example

The `popitem()` method removes the last inserted item (in versions before 3.7, a random item is removed instead):

```
thisdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
thisdict.popitem()  
print(thisdict)
```

Output: {'brand': 'Ford', 'model': 'Mustang'}

Example

The `del` keyword removes the item with the specified key name:

```
thisdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
del thisdict["model"]  
print(thisdict)
```

Output: {'brand': 'Ford', 'year': 1964}

Example

The `clear()` method empties the dictionary:

```
thisdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
thisdict.clear()  
print(thisdict)
```

Output: {}

Loop Through a Dictionary

You can iterate over a dictionary using a for loop. While doing so, the keys of the dictionary are returned, but values can also be returned using methods.

Example

Print all key names in the dictionary, one by one:

```
thisdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
for x in thisdict:  
    print(x)
```

Output:

```
brand  
model  
year
```

Example

You can also use the `values()` method to return values of a dictionary:

```
thisdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
for x in thisdict.values():  
    print(x)
```

Output:

Ford

Mustang

Example

You can use the `keys()` method to return the keys of a dictionary:

```
thisdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
for x in thisdict.keys():  
    print(x)
```

Output:

brand

model

year

Example

Loop through both *keys* and *values*, by using the `items()` method:

```
thisdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
for x, y in thisdict.items():  
    print(x, y)
```

Output:

brand Ford

model Mustang

year 1964

Copy a Dictionary

It's important to note that when you try to copy a dictionary by typing `dict2 = dict1`, it won't create a new copy. Instead, `dict2` will just become a reference to `dict1`. This means that any changes made in `dict1` will also automatically be reflected in `dict2`. To make a true copy of the dictionary, you can use the built-in method called `copy()`.

Example

Make a copy of a dictionary with the `copy()` method:

```
thisdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
mydict = thisdict.copy()  
print(mydict)
```

Output: {'brand': 'Ford', 'model': 'Mustang', 'year': 1964}

Another way to make a copy is to use the built-in function `dict()`

Example

Make a copy of a dictionary with the `dict()` function:

```
thisdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
mydict = dict(thisdict)  
print(mydict)
```

Output: {'brand': 'Ford', 'model': 'Mustang', 'year': 1964}

Nested Dictionaries

A dictionary can contain dictionaries, this is called nested dictionaries.

Example

Create a dictionary that contain three dictionaries:

```
myfamily = {  
    "child1" : {  
        "name" : "Emil",  
        "year" : 2004  
    },  
    "child2" : {  
        "name" : "Tobias",  
        "year" : 2007  
    },  
    "child3" : {  
        "name" : "Linus",  
        "year" : 2011  
    }  
}
```

```
print(myfamily)
```

```
Output: {'child1': {'name': 'Emil', 'year': 2004}, 'child2': {'name':  
'Tobias', 'year': 2007}, 'child3': {'name': 'Linus', 'year': 2011}}
```

Or, if you want to add three dictionaries into a new dictionary:

Example

Create three dictionaries, then create one dictionary that will contain the other three dictionaries:

```
child1 = {  
    "name" : "Emil",  
    "year" : 2004  
}  
child2 = {  
    "name" : "Tobias",  
    "year" : 2007  
}  
child3 = {  
    "name" : "Linus",  
    "year" : 2011  
}
```

```
myfamily = {  
    "child1" : child1,  
    "child2" : child2,  
    "child3" : child3  
}
```

```
print(myfamily)
```

```
Output: {'child1': {'name': 'Emil', 'year': 2004}, 'child2': {'name':  
'Tobias', 'year': 2007}, 'child3': {'name': 'Linus', 'year': 2011}}
```

Access Items in Nested Dictionaries

To access items from a nested dictionary, you use the name of the dictionaries, starting with the outer dictionary:

Example

Print the name of child 2:

```
myfamily = {  
    "child1" : {  
        "name" : "Emil",  
        "year" : 2004  
    },  
    "child2" : {  
        "name" : "Tobias",  
        "year" : 2007  
    },  
    "child3" : {  
        "name" : "Linus",  
        "year" : 2011  
    }  
}  
  
print(myfamily["child2"]["name"])
```

Output: Tobias

Dictionary Methods

Python has a set of built-in methods that you can use on dictionaries.

Method	Description
<code>clear()</code>	Removes all the elements from the dictionary
<code>copy()</code>	Returns a copy of the dictionary
<code>fromkeys()</code>	Returns a dictionary with the specified keys and value
<code>get()</code>	Returns the value of the specified key
<code>items()</code>	Returns a list containing a tuple for each key value pair
<code>keys()</code>	Returns a list containing the dictionary's keys
<code>pop()</code>	Removes the element with the specified key
<code>popitem()</code>	Removes the last inserted key-value pair
<code>setdefault()</code>	Returns the value of the specified key. If the key does not exist: insert the key, with the specified value
<code>update()</code>	Updates the dictionary with the specified key-value pairs
<code>values()</code>	Returns a list of all the values in the dictionary

Sets in Python

In Python programming, a set is a type of collection data that is unordered, iterable, mutable, and free of duplicate elements. Sets are enclosed in curly braces { } and are used to store multiple items in a single variable.

The major advantage of using sets, as opposed to lists, is their highly optimized method for checking if a specific element is contained in the set. This is based on a data structure known as a hash table. However, since sets are unordered, we cannot access items using indexes as we do in lists.

Python has four built-in data types used to store collections of data, namely: list, tuple, dictionary, and set. Each has different qualities and usage.

A set is a collection that is unordered, unchangeable*, and unindexed.

```
myset = {"apple", "banana", "cherry"}
```

❖ Note: Set items are unchangeable, but you can remove items and add new items.

From Python's perspective, sets are defined as objects with the data type 'set':

```
<class 'set'>
```

Example

Create a Set:

```
thisset = {"apple", "banana", "cherry"}  
print(thisset)
```

Output: {'cherry', 'apple', 'banana'}

❖ Note: Sets are unordered, so you cannot be sure in which order the items will appear.

1. Set Items-Set items have three main characteristics. Firstly, they are unordered, which means that the elements in a set are not stored in any particular order. Secondly, they are unchangeable, However, you can still add new items or remove items from it. Finally, sets do not allow duplicate values, which means that each element in a set must be unique.
2. Unordered- Set items in an unordered set have no defined order and cannot be referred to by index or key.
3. Unchangeable- Once a set is created, its items are unchangeable. However, you can still add new items or remove items from it.
4. Duplicates Not Allowed- It is not possible for a set to contain two items with identical values.

Example

Duplicate values will be ignored:

```
thisset = {"apple", "banana", "cherry", "apple"}
```

```
print(thisset)
```

Output: {'banana', 'cherry', 'apple'}

The set() Constructor

It is also possible to use the set() constructor to make a set.

Example

Using the set() constructor to make a set:

```
thisset = set(("apple", "banana", "cherry")) # note the double round-  
brackets  
print(thisset)
```

Output: {'cherry', 'banana', 'apple'}

Access Items

You cannot access items in a set by referring to an index or a key. But you can loop through the set items using a **for** loop or ask if a specified value is present in a set, by using the **in** keyword.

Example

Loop through the set, and print the values:

```
thisset = {"apple", "banana", "cherry"}
```

```
for x in thisset:  
    print(x)
```

Output:

```
cherry  
apple  
banana
```

Example

Check if "banana" is present in the set:

```
thisset = {"apple", "banana", "cherry"}
```

```
print("banana" in thisset)
```

Output: True

❖ Note: Once a set is created, you cannot change its items, but you can add new items.

Get the Length of a Set

To determine how many items a set has, use the `len()` function.

Example

Get the number of items in a set:

```
thisset = {"apple", "banana", "cherry"}
```

```
print(len(thisset))
```

Output: 3

Set Items - Data Types

Set items can be of any data type:

Example

String, int and boolean data types:

```
set1 = {"apple", "banana", "cherry"}
```

```
set2 = {1, 5, 7, 9, 3}
```

```
set3 = {True, False, False}
```

```
print(set1)
```

```
print(set2)
```

```
print(set3)
```

Output:

```
{'cherry', 'apple', 'banana'}
```

```
{1, 3, 5, 7, 9}
```

```
{False, True}
```

A set can contain different data types:

Example

A set with strings, integers and boolean values:

```
set1 = {"abc", 34, True, 40, "male"}
```

```
print(set1)
```

Output: {True, 34, 40, 'male', 'abc'}

Add Set Items

To add one item to a set use the `add()` method.

Example

Add an item to a set, using the `add()` method:

```
thisset = {"apple", "banana", "cherry"}
```

```
thisset.add("orange")
```

```
print(thisset)
```

Output: {'orange', 'cherry', 'banana', 'apple'}

Add Sets

If you want to include items from another set to the existing set, you can use the `update()` method.

Example

Add elements from `tropical` into `thisset`:

```
thisset = {"apple", "banana", "cherry"}  
tropical = {"pineapple", "mango", "papaya"}
```

```
thisset.update(tropical)
```

```
print(thisset)
```

Output: {'apple', 'mango', 'cherry', 'pineapple', 'banana', 'papaya'}

Add Any Iterable

The `update()` method can take any iterable object (e.g., tuples, lists, and dictionaries) as its argument, not just sets.

Example

Add elements of a list to a set:

```
thisset = {"apple", "banana", "cherry"}
```

```
mylist = ["kiwi", "orange"]
```

```
thisset.update(mylist)
```

```
print(thisset)
```

Output: {'banana', 'cherry', 'apple', 'orange', 'kiwi'}

Remove Item

To delete an item from a set, you can make use of two methods: `remove()` and `discard()`.

Example

Remove "banana" by using the `remove()` method:

```
thisset = {"apple", "banana", "cherry"}
```

```
thisset.remove("banana")
```

```
print(thisset)
```

Output: {'cherry', 'apple'}

❖ Note: If the item to remove does not exist, `remove()` will raise an error.

Example

Remove "banana" by using the `discard()` method:

```
thisset = {"apple", "banana", "cherry"}
```

```
thisset.discard("banana")
```

```
print(thisset)
```

Output: {'cherry', 'apple'}

You can alternatively use the `pop()` method to remove an item. However, this method removes a **random item**, so you cannot be sure which item will be removed.

Example

Remove a random item by using the `pop()` method:

```
thisset = {"apple", "banana", "cherry"}
```

```
x = thisset.pop()
```

```
print(x)
```

Output: apple

```
print(thisset)
```

Output: {'banana', 'cherry'}

- ❖ Note: Sets are unordered, so when using the `pop()` method, you do not know which item gets removed.

Example

The `clear()` method empties the set:

```
thisset = {"apple", "banana", "cherry"}
```

```
thisset.clear()
```

```
print(thisset)
```

Output: set()

Example

The `del` keyword will delete the set completely:

```
thisset = {"apple", "banana", "cherry"}
```

```
del thisset
```

```
print(thisset)
```

Output:

Traceback (most recent call last):

File "demo_set_del.py", line 5, in <module>

print(thisset) #this will raise an error because the set no longer exists

NameError: name 'thisset' is not defined

Loop set Items

You can loop through the set items by using a **for** loop:

Example

Loop through the set, and print the values:

```
thisset = {"apple", "banana", "cherry"}
```

```
for x in thisset:  
    print(x)
```

Output:

banana

apple

cherry

Join Two Sets

In Python, you have different options for combining multiple sets. You can use either the `union()` method, which creates a new set containing all elements from both sets, or the `update()` method, which adds all the items from one set into another.

Example

The `union()` method returns a new set with all items from both sets:

```
set1 = {"a", "b", "c"}  
set2 = {1, 2, 3}
```

```
set3 = set1.union(set2)  
print(set3)
```

Output: {'c', 2, 1, 'a', 'b', 3}

Example

The `update()` method inserts the items in set2 into set1:

```
set1 = {"a", "b", "c"}  
set2 = {1, 2, 3}
```

```
set1.update(set2)  
print(set1)
```

Output: {'a', 3, 2, 'b', 1, 'c'}

❖ Note: Both `union()` and `update()` will exclude any duplicate items.

Keep ONLY the Duplicates

If you use the `intersection_update()` method, it will only keep the items that exist in both sets.

Example

Keep the items that exist in both set `x`, and set `y`:

```
x = {"apple", "banana", "cherry"}  
y = {"google", "microsoft", "apple"}
```

```
x.intersection_update(y)
```

```
print(x)
```

Output: {'apple'}

The `intersection()` method returns a new set with common items.

Example

Return a set that contains the items that exist in both set `x`, and set `y`:

```
x = {"apple", "banana", "cherry"}  
y = {"google", "microsoft", "apple"}
```

```
z = x.intersection(y)
```

```
print(z)
```

Output: {'apple'}

Keep All, But NOT the Duplicates

The `symmetric_difference_update()` method retains only the unique elements of both sets.

Example

Keep the items that are not present in both sets:

```
x = {"apple", "banana", "cherry"}  
y = {"google", "microsoft", "apple"}
```

```
x.symmetric_difference_update(y)  
print(x)
```

Output: {'google', 'banana', 'microsoft', 'cherry'}

The `symmetric_difference()` method returns a new set with unique elements present in either of the two sets.

Example

Return a set that contains all items from both sets, except items that are present in both:

```
x = {"apple", "banana", "cherry"}  
y = {"google", "microsoft", "apple"}
```

```
z = x.symmetric_difference(y)  
print(z)
```

Output: {'google', 'banana', 'microsoft', 'cherry'}

- ❖ Note: The values True and 1 are considered the same value in sets, and are treated as duplicates:

Set Methods

Python provides a variety of built-in methods that can be applied to sets.

Method	Description
<code>add()</code>	Adds an element to the set
<code>clear()</code>	Removes all the elements from the set
<code>copy()</code>	Returns a copy of the set
<code>difference()</code>	Returns a set containing the difference between two or more sets
<code>difference_update()</code>	Removes the items in this set that are also included in another, specified set
<code>discard()</code>	Remove the specified item
<code>intersection()</code>	Returns a set, that is the intersection of two other sets
<code>intersection_update()</code>	Removes the items in this set that are not present in other, specified set(s)
<code>isdisjoint()</code>	Returns whether two sets have an intersection or not
<code>issubset()</code>	Returns whether another set contains this set or not
<code>issuperset()</code>	Returns whether this set contains another set or not
<code>pop()</code>	Removes an element from the set
<code>remove()</code>	Removes the specified element
<code>symmetric_difference()</code>	Returns a set with the symmetric differences of two sets
<code>symmetric_difference_update()</code>	inserts the symmetric differences from this set and another
<code>union()</code>	Return a set containing the union of sets
<code>update()</code>	Update the set with the union of this set and others

Thank you